

SUBJECT: Integrating OLTP/OLAP Flows and AI Translation (One-Hot Encoding) into the Dual-Layer Architecture

1. Executive Summary

To successfully predict employee attrition at an MNC scale, our "Dual-Layer Hybrid Architecture" must bridge two completely different worlds: the operational world that records daily HR events (OLTP) and the analytical world that powers dashboards and AI (OLAP). Furthermore, we must translate human-readable text into AI-readable math—a process called One-Hot Encoding.

Because we are executing this in two phases, the way we handle these connections and translations must evolve. This report details exactly how OLTP/OLAP integration and One-Hot Encoding work in Part 1 (Static CSV) versus Part 2 (Enterprise Production).

PART 1: The Static CSV Starting Point (Proof of Concept)

In this phase, we are working with a single, flat CSV file containing 35 attributes (optimized down to 32). There is no live system; the data is frozen in time.

1. How OLTP & OLAP Connect Here:

Technically, a CSV file is a "dead" OLTP system—it represents a single snapshot of transactions with no ability to update.

- **The Connection:** We use Python (PySpark) to artificially simulate the OLTP-to-OLAP pipeline. We take this flat "transactional" dump and programmatically split it into our 5 Logical Themes (Identity, Environment, etc.).
- **The Result:** By splitting the CSV, we create a miniature, static "OLAP" structure (Layer 1). It proves to leadership that grouping data by themes makes dashboards faster and more logical, even before we connect a live database.

2. How One-Hot Encoding Works Here:

AI models cannot read the word "Sales" or "Male"; they only understand numbers. One-Hot Encoding turns text into a simple 1 or 0.

- **The Method:** Because our CSV is static, we already know every possible answer (e.g., we know Departments are only Sales, R&D, and HR).
- **The Execution:** We build our Layer 2 Translation View using simple SQL CASE WHEN logic.
Example: `CASE WHEN Department = 'Sales' THEN 1 ELSE 0 END AS Is_Sales`
- **The Rule:** Layer 1 (Dashboards) keeps the word "Sales" so humans can read it. Layer 2 (AI) turns it into a "1" so the algorithm can read it.

Part 1 Architecture Snapshot:

```
[ Flat CSV File ]  
  | (PySpark splits into 5 Themes)  
  v  
[ LAYER 1: Static OLAP Tables ] -> [ Dashboards (Reads "Sales") ]  
  | (SQL View with CASE WHEN)  
  v  
[ LAYER 2: Virtual Translation ] -> [ ML Model (Reads "1") ]
```

PART 2: The Production Scale (Enterprise System)

In this phase, we move to a live MNC environment (50,000+ employees, daily hires/firings/promotions). We are no longer using Databricks as an all-in-one tool; we are using a split architecture (e.g., Snowflake/BigQuery for storage, Power BI for dashboards, and a separate Python environment for ML).

1. How OLTP & OLAP Connect Here:

This is where true systems integration happens.

- **The OLTP Side:** The HR department uses live software (like Workday or SAP). When an employee is promoted, the HR clerk hits "Save". This system is built for speed, not analysis.
- **The Bridge:** Every night, an automated pipeline extracts only the changes made that day from the OLTP system.
- **The OLAP Side:** The data lands in our Cloud Data Warehouse (e.g., Snowflake). Here, we execute a "Slowly Changing Dimension" (SCD Type 2). If John was promoted, the system does not overwrite his old job title. Instead, it stamps his old record as "Inactive" and creates a brand new row for his promotion. This historical tracking is the fuel our AI needs to learn from past behavior.

2. How One-Hot Encoding Works Here (The Danger & The Fix):

We can no longer use the simple SQL CASE WHEN trick from Part 1.

- **The Problem:** At an enterprise scale, things change. What if the company creates a new department called "AI & Automation" next year? If we hard-coded our SQL view to only look for "Sales", "R&D", and "HR", the new department will break the AI model because it won't be translated into a 1 or 0.
- **The Fix (Dynamic Encoding):** Because we are using a split architecture, the Python ML environment takes over this job. Before training, the Python code queries Layer 2, looks at whatever departments exist today, and automatically generates the 1s and 0s on the fly. If a

new department is added, the Python code automatically creates a new column for it. The model never breaks, and the database admins don't have to rewrite SQL views every time the business changes.

Part 2 Architecture Snapshot:

```
[ LIVE OLTP (Workday/SAP) ]
  | (Nightly automated feed)
  v
[ CLOUD DATA WAREHOUSE (OLAP) ]
  | (SCD Type 2: Preserves history of promotions/role changes)
  v
[ LAYER 1: Theme Tables ] -----> [ POWER BI (Reads "Sales", tracks history) ]
  |
  v
[ LAYER 2: SQL Views (Keeps text as numbers/scales) ]
  | (Data is pulled out to external ML environment)
  v
[ EXTERNAL PYTHON ML ENGINE ]
  | (Dynamically One-Hot Encodes text to 1s/0s)
  | (Trains Model, calculates Flight Risk Score)
  v
[ SCORES PUSHED BACK TO WAREHOUSE ] -> [ POWER BI displays AI Risk Scores ]
```

Conclusion & Strategic Takeaway

By structuring the project into these two parts, we de-risk the implementation.

In Part 1, we use simple, fast SQL tricks to prove that separating human data (Layer 1) from AI data (Layer 2) works perfectly on a static CSV.

In Part 2, we upgrade the plumbing. We connect the live OLTP system to our OLAP warehouse to capture daily history, and we shift the One-Hot Encoding responsibility from rigid SQL to dynamic Python. This ensures that as the MNC grows, changes departments, and adds new job roles over the next 5 years, our AI architecture adapts automatically without requiring constant manual engineering.